백엔드 서비스 조사 보고서

가격 비교 기반 장보기 투두 앱 얼마였지

과목	고급모바일프로그래밍
팀 구성원	2191298 김종현 · 2271494 이재영 · 2311122 신지현
문서 정보	백엔드 조사 v1.0 · 2026년 10월 9일
조사 대상	Firebase · Supabase · AWS Amplify

조사 목적

얼마였지의 장보기 목록과 구매 가격 기록을 저장·동기화할 모바일 백엔드 서비스를 조사한다. 서비스별 특장점과 제약을 비교하고, 프로젝트 핵심 기능의 구현 가능 여부와 방법·예제를 정리한다. 이 자료는 이후 채택기술 보고서의 백엔드 선택 근거로 활용하며 채택 결론은 확정하지 않는다.

핵심 조사 기능

기능	관련 요구사항	조사 내용
목록 동기화와 사용자 구분	F-08~F-12	사용자별 접근, 오프라인 변경 반영, 충돌 선택, 중복 방지
영수증 기반 구매 확정	F-20~F-22	구매분 완료·잔여분 보존, 가격 저장, 부분 실패 복구
구매 가격 조회와 공유	F-23~F-28	본인 이력, 매장별 최근 가격, 공개 정보 제한과 공유 해제

앱과 백엔드의 역할

앱은 사용자 입력을 검증하고 기록을 기기에 먼저 저장한다. 백엔드는 인증된 사용자별 데이터를 구분하고 기기 간 변경을 반영하며 공유 가격을 조회한다. 서버 실패 시 기기 기록과 미전송 요청을 보존하는 조건은 모든 후보에 공통으로 적용한다.

메모·영수증의 문자 인식은 기기에서 처리하고, 사용자가 검토한 텍스트만 서버에 저장한다. 원본 사진 저장·전송 기능은 요구사항에 포함하지 않는다.

조사 기준

모바일 백엔드 서비스 강의 자료의 ‘서비스 2개 이상 비교·핵심 기능 2개 이상 구현 방법 조사’를 기준으로 삼았다. 프로젝트 제안서, 요구사항 분석서 v1.0, UI 프로토타입 설계서 v1.0의 데이터 보존· 계정 구분·구매·조회 흐름을 연결했다.

공식 지원 기능과 요구사항에 따른 구현 구성안을 구분한다. 예제는 설명용 코드이며 실제 서비스 배포·실행 시험 결과를 뜻하지 않는다.

1 서비스별 특장점과 제약

Firebase는 Authentication과 Cloud Firestore, Supabase는 Auth와 PostgreSQL·Realtime, AWS Amplify는 Cognito·AppSync 및 세대별 데이터 연동을 중심으로 비교한다.

항목	Firebase	Supabase	AWS Amplify
데이터 저장	Firestore 문서·컬렉션	PostgreSQL 관계형 테이블	AppSync API와 DynamoDB 등
사용자 구분	Authentication + Security Rules	Auth + RLS	Cognito + API 권한
특장점	모바일 캐시·대기 쓰기와 실시간 조회를 연결	SQL·제약·DB 함수로 관련 데이터를 처리	인증·API·함수를 AWS 자원과 연결
오프라인	모바일 캐시와 변경 동기화 제공	기기 저장·재전송 계층 별도 구성	Gen 1 DataStore 지원; Gen 2 API는 별도 구성
충돌 기본 동작	같은 문서 마지막 쓰기 우선	버전 확인과 충돌 정책을 설계	Gen 1 Auto Merge; 다른 전략 구성 가능
추가 개발 부담	사용자 선택형 충돌과 가격 조회 인덱스 설계	오프라인 대기열·재연결 누락 복구 구현	세대별 지원 확인, 모델·권한·인덱스 관리

공식 근거 [1] [2] [4] [6] [7] [8] [10] [14] [15] [17]

프로젝트 관점의 장단점

Firebase: 모바일 오프라인 처리의 출발점을 제공한다. 다만 기본 마지막 쓰기 우선 정책은 양쪽 변경을 보존하고 사용자가 선택한다는 F-11과 다르므로 추가 로직이 필요하다.

Supabase: 상품·구매·매장 등 관계와 조건별 가격 조회를 SQL로 표현할 수 있다. Realtime만으로 오프라인 편집과 재전송이 완성되지는 않으며 별도 동기화 계층이 필요하다.

AWS Amplify: Gen 1 DataStore의 기기 저장·동기화와 AWS 서버 기능을 연결할 수 있다. Gen 2는 DataStore 지원이 다르므로 두 세대의 기능을 혼합해 판단하면 안 된다.

위 장단점은 요구사항에 비추어 정리한 구현 차이다. 개발 시간·성능·월 비용의 우열은 같은 조건의 실험과 견적 없이 확정하지 않는다.

2 요구사항별 구현 가능 여부

세 서비스는 각 기능의 구현 수단을 제공한다. 아래의 '구성 가능'은 서비스의 기본 설정만으로 모든 요구사항을 충족한다는 뜻이 아니다.

요구 기능	Firebase	Supabase	AWS Amplify
본인 목록 접근 F-08	인증 UID와 Rules로 구성	사용자 ID와 RLS로 구성	Cognito와 소유자 권한으로 구성
기기 간 동기화 F-09~10	캐시·대기 쓰기와 상태 안내	로컬 대기열 + RPC·Realtime	Gen 1 DataStore 또는 Gen 2 API + 대기열
충돌 선택 F-11	버전 비교·후보 보존 추가	버전 조건 수정·후보 보존 추가	충돌 전략·후보 보존·선택 UI 추가
중복 방지 F-12	요청 ID 결과를 원자적으로 기록	요청 ID 고유 제약과 DB 함수	요청 ID와 조건부·트랜잭션 처리
구매 확정 F-20~22	서버 트랜잭션 구성 가능	DB 트랜잭션 RPC 구성 가능	사용자 정의 mutation 구성 가능
가격·공유 F-23~28	공개 문서·조건 조회 구성	공개 테이블·SQL 조회 구성	공개 모델·인덱스 조회 구성

구현 수단 근거 [2] [3] [4] [7] [8] [10] [11] [15] [17] [19]

공통 데이터 구성안

장보기 항목은 항목 ID·사용자 ID·상품명·수량·완료 상태·버전·삭제 표시를 가진다. 가격 기록은 구매 ID·상품·규격·할인 조건·매장·구매일·수량·실제 결제 금액을 가진다. 전송 요청은 요청 ID·사용자 ID·기준 버전·변경 내용·처리 상태를 보관한다.

버전은 사용자가 수정한 기준 상태와 현재 서버 상태를 비교하는 값이다. 삭제 표시를 남기면 오래된 기기가 같은 항목을 다시 생성하는 상황을 구분할 수 있다. 삭제 표시의 정리 시점은 장기간 오프라인인 기기의 복귀까지 고려해 정한다.

기기와 서버 사이의 저장 순서

> 입력 검증 → 기기 저장 → 화면 반영 → 서버 반영 → 서버 확인 후 대기 요청 정리

기기 저장 실패 시 완료로 표시하지 않는다. 계정 전환 후 이전 계정의 요청·응답을 새 계정과 합치지 않는다. 동기화 대기·진행·완료·실패와 미해결 충돌을 구분한다.

3 Firebase 목록 동기화 구현

기본 지원과 추가 구현

Firestore는 모바일에서 캐시 읽기와 오프라인 쓰기를 지원하고 재연결 후 변경을 반영한다. 같은 문서의 여러 변경은 기본적으로 마지막 쓰기가 우선하므로 사용자 선택형 충돌과는 다르다. 캐시에 있는 데이터와 모든 기록의 영구 보관도 구분한다. [2]

구현 구성안: 항목마다 버전을 보관하고 변경 요청의 기준 버전과 서버 버전을 비교한다. 충돌이면 로컬 변경과 서버 값을 모두 저장해 선택 화면을 연다. 삭제도 버전이 있는 삭제 표시로 전파한다. 재전송은 같은 요청 ID를 사용한다.

온라인 반영 시 버전 확인 예시

```
// 인증된 사용자의 항목 참조와 검증된 수량을 전달한다.
Future<void> updateQuantity(
  DocumentReference<Map<String, dynamic>> itemRef,
  int expectedVersion,
  int quantity,
) async {
  if (quantity < 1 || quantity > 999) {
    throw ArgumentError('수량 범위 오류');
  }
  await FirebaseFirestore.instance.runTransaction((tx) async {
    final snapshot = await tx.get(itemRef);
    final item = snapshot.data();
    if (item == null || item['version'] != expectedVersion ||
        item['deleted'] == true) {
      throw StateError('충돌 또는 삭제 확인 필요');
    }
    tx.update(itemRef, {
      'quantity': quantity,
      'version': expectedVersion + 1,
    });
  });
}
```

cloud_firestore 초기화와 import를 생략한 설명용 코드다. 트랜잭션은 오프라인에서 실패하므로 로컬 대기열을 저장한 뒤 온라인에서 호출하는 구성이다. 콜백은 재실행될 수 있으므로 그 안에서 화면·로컬 상태를 바꾸지 않는다. [3]

사용자 접근과 중복 요청

개인 목록은 users/{uid}/items/{itemId}처럼 구분하고 Security Rules에서 인증 UID와 경로를 대조한다. 소유자만 검사해서는 충분하지 않으므로 입력 범위와 버전 증가도 검증한다. 서버 SDK를 사용하는 API는 자체 인증·권한 검사가 필요하다. [1] [4]

예시는 요청 ID 저장을 생략했다. 응답 유실 후 재전송까지 처리하려면 처리 결과 문서를 같은 서버 트랜잭션에 저장하고 동일 요청의 기존 결과를 반환해야 한다.

4 Supabase 목록 동기화 구현

Supabase는 Flutter 클라이언트, PostgreSQL과 사용자 인증을 제공한다. Realtime의 DB 변경 구독은 온라인 변경을 받는 수단이다. 앱 재실행 후 오프라인 변경을 반영하는 저장·전송 대기열까지 같은 기능으로 간주하면 안 된다. [6] [8]

구현 구성안: SQLite에 항목과 대기 요청을 저장하고, 재연결 시 RPC로 기준 버전을 확인한다. Realtime은 새 변경 알림에 활용하며 연결 복구 시 누락 구간을 재조회한다. Brick 등 별도 오프라인 계층을 연동하는 방법도 공식 자료에 소개되어 있다. [9] [11]

RLS와 버전 조건 수정 예시

```
-- items 테이블과 기본 CRUD 권한이 준비된 설명용 예시다.
alter table public.items enable row level security;
create policy own_items on public.items
  for all to authenticated
  using ((select auth.uid()) = owner_id)
  with check ((select auth.uid()) = owner_id);

create function public.update_item_quantity(
  p_id uuid, p_version bigint, p_quantity integer
) returns setof public.items
language sql security invoker set search_path = '' as $$
  update public.items
  set quantity = p_quantity, version = version + 1
  where id = p_id and owner_id = auth.uid()
    and version = p_version and deleted = false
    and p_quantity between 1 and 999
  returning *;
$$;
revoke execute on function public.update_item_quantity(
  uuid, bigint, integer) from public, anon;
grant execute on function public.update_item_quantity(
  uuid, bigint, integer) to authenticated;
```

테이블 생성, CHECK 제약, 오류 응답 모델, 요청 중복 기록은 생략했다. 함수는 호출자 권한으로 실행한다. 반환 행이 없으면 유효성 오류·삭제·충돌·접근 제한을 확인하며, 0건을 동기화 성공으로 처리하지 않는다. [7] [10]

데이터 보존과 계정 변경

RLS는 서버 행 접근을 제한하고 로컬 DB의 계정 분리는 앱이 담당한다. 로그아웃 시 이전 계정의 전송을 멈추고 계정별 미전송 내용을 보존한다. 공유 조회는 개인 테이블 전체를 공개하지 않고 허용된 필드만 반환하는 API로 구성한다.

5 AWS Amplify 목록 동기화 구현

Gen 1 DataStore와 Gen 2 구분

Gen 1 DataStore는 기기 저장소와 AppSync 동기화를 연결한다. 기본 Auto Merge는 단일 필드에서 서버 값을 우선하는 방식이며, Optimistic Concurrency나 Custom Lambda 등 다른 전략도 구성할 수 있다. [14] [15]

Gen 2의 Flutter 기능표는 DataStore와 해당 충돌 감지를 지원하지 않는 것으로 구분한다. 공식 이전 가이드는 Amplify API와 별도 로컬 캐시·오프라인 계층을 사용하는 방법을 설명한다. 따라서 AWS 후보는 'Gen 1 DataStore 구성'과 'Gen 2 API 구성'을 분리해 판단해야 한다. [17] [18]

Gen 1 목록 모델과 로컬 저장 예시

```
# Gen 1 GraphQL 모델 예시
type ShoppingItem @model @auth(rules: [{allow: owner}]) {
  id: ID!
  title: String!
  quantity: Int!
  completed: Boolean!
}
```

```
// 위 모델의 Dart 코드 생성과 Amplify 초기화 이후 호출한다.
await Amplify.DataStore.save(ShoppingItem(
  id: stableItemId,
  title: validatedTitle,
  quantity: validatedQuantity,
  completed: false,
));
```

데이터 조작 API를 보여 주는 설명용 코드다. stableItemId는 첫 생성 후 유지하는 항목 ID이며 검증값은 호출 전에 준비한다. @auth 구성은 해당 세대의 인증·모델 생성 설정과 함께 적용한다. save 완료를 모든 기기의 동기화 완료로 표시하지 않는다. [16] [17]

사용자 선택형 충돌 구성안

자동 병합 대신 충돌을 검출하는 전략을 구성하고, 앱의 별도 저장 영역에 두 후보와 원래 요청을 보존한다. conflictHandler 등에서 충돌을 수집해 화면에 전달하고 사용자가 선택한 뒤 다시 반영한다. 화면에서 선택할 때까지 장기 보존·삭제 충돌·중복 요청 상태를 유지하는 것은 추가 구현 범위다. [15]

Gen 2 API 구성에서는 SQLite·전송 대기열·버전 검증 mutation을 따로 구성할 수 있다. SDK와 백엔드 세대를 함께 기록해야 이전 예제를 현재 서비스의 기본 지원으로 오해하지 않는다.

6 구매 확정과 중복 방지 구현

핵심 기능 2: F-20~F-22는 목록 상태와 가격 기록이 함께 남는 구매 확정을 요구한다. 우유 3개 중 2개를 구매하면 2개 완료와 가격 기록을 남기고 1개 미완료를 보존한다. 아래 내용은 요구사항에 따른 서비스별 구현 구성안이다.

서비스	서버에서 함께 처리할 방법	추가로 보장할 조건
Firebase	트랜잭션으로 요청 ID 확인, 항목 버전 확인, 잔여·완료 항목과 가격 기록 저장	모든 읽기를 쓰기 전에 수행. 충돌이면 전체 거부. 동일 요청은 기존 결과 반환
Supabase	DB 함수 RPC의 한 트랜잭션에서 소유자·버전 확인과 목록·가격 저장	사용자+요청 ID 고유 제약. 동시 요청 직렬화·행 잠금 또는 조건 수정. 오류 시 롤백
AWS Amplify	사용자 정의 AppSync mutation과 DynamoDB 트랜잭션 등으로 다중 기록 저장	DataStore.save를 여러 번 호출한 것만으로 전체 구매 원자성을 가정하지 않음

원자적 서버 처리의 기술 근거 [3] [10] [13] [19]

공통 처리 흐름 예시

```
구매 확정(사용자, 요청 ID, 변경 묶음)  // 설명용 의사코드
  서버 트랜잭션 시작
  인증과 소유권 확인
  같은 요청의 저장 결과가 있으면 기존 결과 반환
  요청 ID를 예약하고 동일 요청의 동시 실행을 제어
  연결한 목록 항목들의 기준 버전과 현재 버전 확인
  충돌 또는 필수값 오류이면 전체 롤백
  구매분 완료와 잔여분, 가격 기록을 함께 저장
  동일 요청의 처리 결과 저장
  서버 트랜잭션 커밋
```

응답 유실과 영수증 중복의 차이

서버 저장 후 응답만 유실되면 같은 요청 ID로 재시도한다. 같은 요청 ID에 다른 내용을 보내면 오류로 처리하도록 내용 해시 등의 검증이 필요하다. 구매 건마다 안정적인 ID를 사용하되, ID 고정만으로 수정·완료 부작용의 중복까지 방지되는 것은 아니다.

다시 촬영한 영수증은 새 요청이므로 매장·구매일·상품·수량·금액으로 기존 구매 후보를 찾는다. 사용자가 별개 구매임을 확인했을 때만 새 기록을 만든다. 이 판정 정책은 D-04의 확정 대상이다.

서버 트랜잭션은 기기의 로컬 저장과 한 트랜잭션이 아니다. 로컬 확정 후에는 대기 상태를 유지하고, 서버 확인 뒤에만 동기화 완료로 바꾼다.

7 Firebase와 Supabase 가격 조회

핵심 기능 3: F-23~F-28은 본인 이력과 다른 사용자의 매장별 최근 구매 가격을 조회한다. 최근 기록은 구매일 내림차순, 같은 구매일이면 등록 시점 내림차순으로 정한다. 상품·규격·할인 조건이 다른 기록은 비교 순위에서 제외한다.

Firebase 조회 구성안

개인 기록과 공개 가격을 분리하고 공개 컬렉션에는 계정 정보를 넣지 않는다. 같은 비교 조건과 매장으로 필터링한 뒤 구매일·등록 시점을 정렬할 수 있다. 조합에 맞는 인덱스가 필요하다. [5]

```
// 한 매장에 대한 공개 가격 후보를 조회하는 Dart 예시다.
final latest = await FirebaseFirestore.instance
    .collection('public_prices')
    .where('comparisonKey', isEqualTo: comparisonKey)
    .where('storeId', isEqualTo: storeId)
    .orderBy('purchasedOn', descending: true)
    .orderBy('registeredAt', descending: true)
    .limit(1)
    .get();
```

공개 레코드만 이 컬렉션에 존재한다는 가정이다. 매장 목록과 인덱스 설정은 생략했다. 매장이 많아지면 매장별 최근 기록을 서버에서 관리하는 별도 집계 구성도 검토할 수 있다. 공유 해제 시 공개 사본 삭제와 최신 기록 재선정을 함께 처리해야 한다.

Supabase 조회 구성안

SQL의 DISTINCT ON으로 동일 조건에서 매장별 첫 기록을 고를 수 있다. 공개 응답은 SELECT 열을 명시하며 작성자 ID는 반환하지 않는다. [12]

```
-- 공개 가격 전용 테이블을 가정한 SQL 예시다.
select distinct on (store_id)
  public_id, store_id, product_name, purchased_on,
  package_size, unit, quantity, paid_amount, discount_key
from public.shared_prices
where comparison_key = :comparison_key
order by store_id, purchased_on desc, registered_at desc;
```

:comparison_key는 설명용 바인딩 변수다. 실제 DB 함수에서는 인자로 받는다. 테이블의 열 제약·공개 정책·읽기 권한 설정은 별도로 필요하다. 임의 권한의 DB 뷰를 만들었다고 개인 데이터 보호가 자동 적용된다고 가정하지 않는다.

8 AWS 가격 조회와 공개 정보 관리

AWS Amplify 가격 조회 구성안

공개 가격 모델을 개인 구매 모델과 분리한다. DynamoDB 기반 구성에서는 비교 조건과 매장을 조회 키로 만들고 구매일·등록 시점을 정렬 키에 포함하는 인덱스를 설계할 수 있다. AppSync에서 최신 1건 조회 API를 제공하고 매장별 결과를 묶어 반환한다.

처리	구현 방법	확인할 조건
본인 이력	소유자 권한 모델을 구매일순으로 조회	다른 사용자의 개인 기록 접근 차단
매장별 최근 기록	비교 조건·매장 인덱스로 최신 기록 조회	구매일 우선, 같은 날짜면 등록 시점 우선
공개 API	허용된 가격 필드만 담은 모델·응답 반환	owner 등 내부 인증·작성자 식별 필드 제외
공유 해제	공개 기록 제거 후 최근 결과 재선정	캐시 갱신과 이전 응답 반영 제어

모델·인덱스·사용자 정의 query 근거 [17]

조회 처리 예시

```
매장별 최근 가격 조회(비교 조건, 매장 목록)  // 의사코드
  요청자의 조회 권한과 입력 조건 확인
  같은 상품·규격·할인 조건의 공개 기록만 조회
  각 매장에서 구매일과 등록 시점이 가장 최근인 기록 선택
  공개 ID·상품·매장·구매일·규격·수량·금액만 반환
  기록 없음과 조회 실패를 구분해 반환
```

실제 GraphQL query나 resolver 코드는 데이터 모델·인덱스가 확정된 뒤 작성한다. 이는 구현 방법을 설명하는 의사코드이며 기본 제공 최저가 API를 뜻하지 않는다.

세 서비스에 공통으로 필요한 공개 정책

공개 가격의 필드는 상품·매장·구매일·용량·단위·수량·가격·할인 조건으로 제한한다. 개인 목록·계정 식별 정보·결제 정보·사진은 반환하지 않는다. 공유 여부만 검사하고 개인 문서나 행 전체를 반환하는 방식은 충분하지 않다.

공유 해제 시 개인 가격 이력은 유지하고 공개 조회 대상에서 제외한다. 공개 사본을 별도로 저장하면 원본 변경·삭제·공유 해제와 사본 상태를 함께 관리해야 한다. 최신 기록을 해제했다면 이전의 공개 기록을 다시 선정한다.

표시 금액은 과거 구매 기록이다. 서버 조회가 성공했더라도 현재 판매가나 재고를 보장하지 않는다. 상품·규격·할인 조건이 다르거나 확인 불가이면 비교 순위에서 제외한다.

9 비용과 유지보수 조건

서비스 비용은 무료 여부만으로 비교하기 어렵다. 사용자 수, 기기 수, 동기화 빈도, 조회되는 기록 수와 서버 함수 호출량을 같은 가정으로 놓아야 한다. 아래는 가격표의 과금 요소를 정리한 것으로 특정 요금이나 월 비용을 확정한 견적은 아니다.

서비스	주요 과금 요소	이 앱에서 늘어날 수 있는 사용량
Firebase	Firestore 읽기·쓰기·삭제, 저장·전송, 함수와 인증 방식별 비용	실시간 구독과 재조회, 공개 가격 갱신, 구매 트랜잭션 재시도
Supabase	요금제·DB 컴퓨트, 저장·전송, 활성 사용자·Realtime 등 할당량	가격 조회와 DB 함수, 기기별 변경 구독, 별도 동기화 서비스
AWS Amplify	Cognito·AppSync·DynamoDB·Lambda 등 구성한 AWS 서비스별 과금	구독·API 요청, 트랜잭션, 인덱스와 함수 실행

공식 요금 정보 [20] [21] [22] [23]

공통 견적 입력 예시

예를 들어 월 사용자 100명·사용자당 기기 2대·하루 목록 변경 10건·하루 가격 조회 5회처럼 공통 가정을 정할 수 있다. 이 수치는 조사용 예시이며 실제 예상 수요가 아니다. 한 화면 조회에 읽는 레코드 수, 동기화 때 전송하는 변경 수와 재시도율을 추가해 사용량으로 환산한다.

원본 사진을 업로드하지 않으므로 사진 저장 비용은 필수 기능 견적에 포함하지 않는다. 공개 가격의 중복 저장, 재접속 구독, 인덱스 읽기, 별도 오프라인 연동 서비스 비용은 선택한 구성에 따라 달라진다. 지역·요금제·무료 할당 조건은 채택 시점의 공식 가격표로 다시 계산한다.

플랫폼 종속성과 유지보수

Firestore의 문서·규칙·쿼리, Supabase의 SQL·RLS·RPC, Amplify의 모델 생성·AppSync 권한·AWS 인프라 설정은 각각 다른 이전 비용을 만든다. PostgreSQL 사용만으로 인증과 실시간 API까지 그대로 이전되는 것은 아니다.

인증·권한 규칙, 서버 함수, 데이터 모델·인덱스의 버전을 함께 관리한다. 계정 전환·데이터 이전·장기간 오프라인 복귀 시험은 백엔드 유지보수 조건이다.

10 조사 결과와 후속 확인

조사 결과

Firebase는 모바일 캐시·대기 쓰기, Supabase는 SQL·RLS·DB 함수, AWS Amplify는 인증·API·AWS 자원 연동과 세대별 데이터 기능을 제공한다. 세 서비스 모두 얼마였지의 사용자 구분·목록 동기화·구매 기록·가격 조회를 구성할 수 있으나 충돌 선택과 구매 일관성·중복 방지는 추가 설계가 필요하다.

목록 동기화는 같은 항목의 기준 버전 비교와 양쪽 변경 보존, 구매 확정은 서버 트랜잭션과 요청 ID 처리, 가격 조회는 동일 조건 필터와 구매일 우선 정렬·공개 필드 제한이 핵심이다. 이 결과는 기술 채택 결론과 구분하며 후보별 성능이나 비용 우열을 확정하지 않는다.

구현 후 확인할 시험

항목	시험 방법	통과 조건
접근 통제	다른 계정 목록 조회·수정, 계정 전환	본인 자료만 접근, 이전 계정 대기 변경 분리
동기화	두 기기 단절·재연결, 같은 요청 3회 전송	누락·중복·삭제 항목 부활 없음
충돌	같은 항목 수정, 수정·삭제, 전송 중 새 변경	양쪽 보존, 선택 후 두 기기 상태 일치
구매 확정	부분 구매, 서버 실패, 저장 후 응답 유실	목록·가격 상태 일치, 중복 구매 기록 없음
가격·공유	구매일·등록일 역전, 조건 불일치, 공유 해제	매장별 최근 구매 기록, 공개 필드 제한

채택 전에 팀에서 확정할 항목

D-01의 BaaS와 시험 환경, D-02의 사용자 확인 방식, D-03의 충돌·중복 이름 처리, D-04의 부분 구매·재등록, D-05의 공유·삭제 정책을 확정한다. 확정한 정책을 후보별 구현 시연·실패 흐름 시험과 동일 조건 비용 산정에 적용한다.

요구사항의 동기화 성능 제안은 정상 네트워크에서 변경 100건을 재연결 후 10초 이내 반영하거나 충돌로 표시하는 것이다. 가격 조회는 정상 조건에서 5초 이내 결과, 지연 시 10초 이내 실패 안내를 제안한다. 이는 이번 조사에서 측정한 값이 아니다.

본 문서는 조사 단계 산출물이다. SDK 예제 실행, 실제 서비스 배포, 요구사항 시험 통과와 최종 기술 채택은 이후 검증 결과로 판단한다.

11 조사 자료와 공식 출처

프로젝트와 강의 자료

모바일 백엔드 서비스 강의 자료 8쪽 팀 활동, 모바일 앱 프로젝트 제안서, 얼마였지 요구사항 분석서 v1.0 HM-REQ-001, UI 프로토타입 설계서 v1.0을 기준으로 삼았다.

공식 출처

2026년 10월 9일 확인. 본문 인용 번호에 직접 링크를 연결했다. 제품 문서와 요금은 갱신될 수 있으므로 채택 시점의 조건을 다시 확인한다.

[1] Firebase Authentication Flutter 시작
https://firebase.google.com/docs/auth/flutter/start

[2] Firestore 오프라인 데이터 접근
https://firebase.google.com/docs/firestore/manage-data/enable-offline

[3] Firestore 트랜잭션과 일괄 쓰기
https://firebase.google.com/docs/firestore/manage-data/transactions

[4] Firestore Security Rules 조건
https://firebase.google.com/docs/firestore/security/rules-conditions

[5] Firestore 단순 및 복합 쿼리
https://firebase.google.com/docs/firestore/query-data/queries

[6] Supabase Flutter 시작
https://supabase.com/docs/guides/getting-started/quickstarts/flutter

[7] Supabase Row Level Security
https://supabase.com/docs/guides/database/postgres/row-level-security

[8] Supabase Postgres Changes
https://supabase.com/docs/guides/realtime/postgres-changes

[9] Supabase와 Flutter 및 Brick 오프라인 구현
https://supabase.com/blog/offline-first-flutter-apps

[10] Supabase Database Functions
https://supabase.com/docs/guides/database/functions

[11] Supabase Flutter RPC
https://supabase.com/docs/reference/dart/rpc

12 공식 출처 API와 비용

2026년 10월 9일 확인. 본문 인용 번호에 직접 링크를 연결했다. 제품 문서와 요금은 갱신될 수 있으므로 채택 시점의 조건을 다시 확인한다.

[12] PostgreSQL SELECT
https://www.postgresql.org/docs/current/sql-select.html

[13] PostgreSQL Transactions
https://www.postgresql.org/docs/current/tutorial-transactions.html

[14] Amplify Flutter Gen 1 DataStore
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/

[15] Amplify Flutter DataStore 충돌 처리
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/conflict-resolution/

[16] Amplify Flutter DataStore 데이터 조작
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/manipulate-data/

[17] Amplify Flutter Gen 2 Feature Parity
https://docs.amplify.aws/flutter/start/migrate-to-gen2/feature-matrix/

[18] Amplify Flutter DataStore에서 이전
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/migrate-from-datastore/

[19] AppSync DynamoDB 트랜잭션
https://docs.aws.amazon.com/appsync/latest/devguide/tutorial-dynamodb-transact.html

[20] Firebase 요금제
https://firebase.google.com/pricing

[21] Firestore 과금 구조
https://firebase.google.com/docs/firestore/pricing

[22] Supabase 요금제
https://supabase.com/pricing

[23] AWS Amplify 요금 구조
https://aws.amazon.com/amplify/pricing/